

A REPORT OF FOUR WEEK TRAINING

At

GURU NANAK DEV ENGINEERING COLLEGE

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD

OF THE DEGREE OF

BACHELOR OF TECHNOLOGY

(Information Technology)



JUNE-JULY ,2025

SUBMITTED BY:

NAME: Alisha

UNIVERSITY ROLL NO: 2203792

DEPARTMENT OF INFORMATION TECHNOLOGY

GURU NANAK DEV ENGINEERING COLLEGE LUDHIANA

(An Autonomous College Under UGC ACT)

CONTENTS

TOPIC	Page no.
<i>Certificate</i>	<i>i</i>
<i>Candidate's declaration</i>	<i>ii</i>
<i>Abstract</i>	<i>iii</i>
<i>Acknowledgement</i>	<i>iv</i>
<i>About GNDEC Ludhiana</i>	<i>v</i>
CHAPTER 1 INTRODUCTION	
1.1 Introduction to Prompt Engineering	1
1.2 Need for Prompt Engineering in the Modern World	2
1.3 Evolution of Prompt Engineering	2
1.4 Software and Tools Used in Prompt Engineering	3
1.5 Significance of Prompt Engineering	4
1.6 Advantages of Prompt Engineering	5
CHAPTER 2 TRAINING WORK UNDERTAKEN	
2.1 Prompt Engineering	6-7
2.2 Generative AI	8-9
2.3 Large Language Model (LLM)	10-11
2.4 Playground	12
2.5 Prompt Evaluation Techniques	13
2.6 LangChain	14
2.7 RAG (Retrieval-Augmented Generation)	15
2.8 Notebook LM (Google AI Tool)	16
2.9 Langflow	17
2.10 Methodology Followed	17
CHAPTER 3 RESULTS AND DISCUSSIONS	
3.1 Summary Generation Results	18
3.2 AI Chat Interaction	19
3.3 PDF Generation Feature – Code Explanation	20-23

3.4 Discussion & Insights	24
---------------------------	----

CHAPTER 4 CONCLUSION AND FUTURE SCOPE

4..1 Conclusion and future scope	25
References	26



GURU NANAK DEV ENGINEERING COLLEGE

An Autonomous College under UGC Act 1956

GILL PARK, GILL ROAD, LUDHIANA - 141006 (PUNJAB) INDIA

AICTE Approved, ISO: 9001-2015 Certified, Affiliated to IK Gujral, PTU, Kapurthala
IEI Accredited, UG Programmes, Institute Accredited by NAAC ('A' Grade) & TCS

TESTING AND CONSULTANCY CELL

Website : <http://tcc.gndec.ac.in>

E-mail : tcc@gndec.ac.in

This is certified that Ms./Mr. Alisha D/o, S/o S. Sandeep Singh

Roll No. 2203792 of college Guru Nanak Dev Engineering College, Ludhiana

branch Civil Engg. / Mechanical Engg. / Electronics & Communication Engg. / Electrical Engg. /

Computer Science & Engg. / Information Technology has successfully completed participated Industrial

Training / Institutional Training Programme on PLC Engineering

from June 23, 2025 to July 21, 2025


Co-ordinator


Head of the Department


Dean (T & C)


Principal

CANDIDATE'S DECLARATION

I “ALISHA” hereby declare that I have undertaken four-week training “GURU NANAK DEV ENGINEERING COLLEGE” during a period from _____ to _____ in partial fulfilment of requirements for the award of degree of B. Tech (Information Technology) at GURU NANAK DEV ENGINEERING COLLEGE, LUDHIANA. The work which is being presented in the training report submitted to Department of Information Technology Engineering at GURU NANAK DEV ENGINEERING COLLEGE, LUDHIANA is an authentic record of training work.

Signature of the Student

The four-week industrial training Viva–Voce Examination of _____ has been held on _____ and accepted.

Signature of Internal Examiner

Signature of External Examiner

Abstract

Prompt Engineering is an emerging discipline in the field of Artificial Intelligence that focuses on designing effective prompts to optimize the responses generated by Large Language Models (LLMs) such as GPT and Gemini. This report explores various foundational concepts related to prompt engineering, including its definition, evolution, and importance in modern AI applications. It examines different types of prompts such as zero-shot, few-shot, role-based, chain-of-thought, instruction, constraint-based, and reframing prompts, each serving a unique purpose in controlling the model's output. The study also highlights the difference between prompt engineering and traditional programming, demonstrating how prompt engineering leverages natural language instructions rather than rigid code. Additionally, advanced AI technologies and tools such as LangChain, Langflow, Retrieval-Augmented Generation (RAG), and Notebook LM are discussed to show how prompt engineering is integrated into real-world systems. The report further explains how LLMs function using key terms like tokens, parameters, and context windows, along with their applications in education, healthcare, business automation, creative industries, and research. Moreover, the limitations of LLMs, such as hallucinations, dependency on training data, lack of real-time awareness, and ethical concerns, are addressed. Overall, the abstract emphasizes how prompt engineering plays a crucial role in enhancing human-AI interaction, improving response accuracy, and enabling the development of intelligent, personalized AI solutions. This report provides a comprehensive overview of prompt engineering as a transformative skill in today's AI-driven world and its potential to redefine the future of automation and decision-making systems.

ACKNOWLEDGEMENT

I am deeply grateful to my training mentor and faculty for providing me with the opportunity to explore the emerging field of Prompt Engineering. Their valuable guidance, encouragement, and continuous support helped me gain a deeper understanding of advanced AI concepts such as prompt design, large language models, retrieval-augmented generation (RAG), LangChain, and generative AI tools. I also extend my sincere thanks to the institution and training department for creating a supportive learning environment that enabled me to apply theoretical knowledge to practical implementation.

Lastly, I would like to thank my peers, friends, and family for their constant motivation and cooperation throughout this learning journey. This report is a result of their inspiration, and I am thankful to everyone who directly or indirectly contributed to its successful completion.

About the Institute

Guru Nanak Dev Engineering College (GNDEC), Ludhiana, is one of the premier engineering institutions in northern India. It was established in 1956 by the Nankana Sahib Education Trust (NSET) in memory of the birthplace of Guru Nanak Dev Ji, the founder of Sikhism. The foundation stone of the college was laid by Dr. Rajendra Prasad, the first President of India. The institute is located at Gill Park, Ludhiana, Punjab, and spreads over 88 acres of lush green campus equipped with modern infrastructure and excellent academic facilities. It is affiliated with I.K. Gujral Punjab Technical University (IKGPTU), Jalandhar, and approved by the All-India Council for Technical Education (AICTE), New Delhi. The college is also accredited by NAAC with an 'A' Grade, reflecting its commitment to academic excellence. GNDEC offers a wide range of Undergraduate (B. Tech), Postgraduate (MTech, MBA, MCA), and Doctoral (Ph.D.) programs in various engineering disciplines. The curriculum is designed to combine theoretical concepts with practical exposure, helping students develop a solid foundation in engineering and technology. The college has well-equipped laboratories, advanced computer centres, workshops, a rich central library, hostels, sports facilities, and a serene campus environment that fosters holistic development. The faculty at GNDEC is highly qualified and dedicated to nurturing students through innovative teaching methods, research, and project-based learning. The Training and Placement Cell actively support students in securing placements with reputed companies such as Infosys, TCS, HCL, and Accenture.

CHAPTER 1

INTRODUCTION

1.1 Introduction to Prompt Engineering

Prompt engineering is the process of designing, optimizing, and structuring prompts (inputs or instructions) in a way that enables AI models to understand user intent clearly and generate accurate responses. Instead of programming through code, users now interact with AI using natural language. The quality of output entirely depends on how well the prompt is written. Effective prompt engineering ensures that the AI model follows user instructions, produces relevant content, adheres to constraints like tone and word limit, and solves problems efficiently. Prompt engineering involves techniques such as zero-shot prompting, few-shot prompting, role-based prompting, chain-of-thought prompting, and constraint-based prompting. These techniques help control the model's output and guide its reasoning process.

1.2 Need for Prompt Engineering in the Modern World

In today's world, industries are shifting towards automation and intelligent systems that can think, analyse, and generate solutions. Prompt engineering enables non-programmers, students, researchers, and professionals to leverage AI models without deep coding knowledge. It enhances productivity, reduces manual effort, and increases the accuracy of AI-generated responses. Businesses use prompt engineering to improve customer service, create marketing content, analyse data, and make informed decisions. Moreover, prompt engineering is a critical skill in the era of Generative AI, where AI is used not only for answering questions but also for creating text, images, audio, and videos.

1.3 Evolution of Prompt Engineering

Initially, AI systems were rule-based and could only perform tasks defined by programmers. With the introduction of machine learning, AI could learn from data, but still required large datasets and complex programming.

The major breakthrough came with Large Language Models, which can understand and generate human-like responses. Early models responded only to simple inputs. However, as AI models became more advanced, users realized that the style and structure of the input (prompt) greatly influenced the output.

This marked the birth of prompt engineering. Over time, techniques such as:

- Zero-shot prompting (no examples),
- Few-shot prompting (with examples),
- Role-based prompting (assigning roles to AI),
- Chain-of-thought prompting (logical reasoning)

were introduced to improve performance. Today, prompt engineering is a formal discipline essential for working with AI.

1.4 Software and Tools Used in Prompt Engineering

During this training, several AI tools and platforms were studied, including:

1. OpenAI Playground – Used to interact with GPT models and test prompt behaviour.
2. Google AI Studio (Gemini Playground) – A platform for working with Gemini models using text or multimodal prompts.
3. LangChain – A programming framework that connects LLMs with external tools and

databases.

4. Langflow – A no-code visual tool based on LangChain for building AI workflows.

1.5 Significance of Prompt Engineering

Prompt engineering is not just a technical skill; it is a strategic skill that allows individuals and organizations to harness the true power of AI. It enables:

- Improved accuracy in AI outputs
- Time saving in tasks such as writing, coding, and data analysis
- Enhanced user experience in AI-based applications
- Cost reduction through automation

1.6 Advantages of Prompt Engineering

○ Improves Accuracy of AI Output

Well-crafted prompts help AI produce precise, relevant, and high-quality results.

○ Enhances Control Over AI Behavior

Users can guide the tone, style, depth, and format of responses using prompt instructions.

○ Increases Productivity

Saves time by enabling fast generation of content, code, ideas, solutions, etc.

○ Reduces Need for Complex Programming

Instead of writing large code, users can achieve outputs with simple language prompts.

Disadvantages of Prompt Engineering

- **Requires Skill and Experimentation**

Effective prompting is not always straightforward. It often needs multiple trials to get the right output.

- **Inconsistent Results**

The same prompt may give different responses each time depending on the model's interpretation.

- **Bias and Hallucinations**

AI may generate factually incorrect or biased information even if the prompt is well-written.

- **Dependent on Model Limitations**

If the underlying LLM is weak or outdated, even the best prompt won't produce good results.

CHAPTER 2

TRAINING WORK UNDERTAKEN

2.1 Prompt Engineering

Prompt Engineering is the process of designing and improving prompts so that AI models can produce accurate, relevant, and well-structured responses.

2.1.1 Type of Prompts

1. Zero-Shot Prompt

A prompt where no examples are provided. The model responds only based on instruction and its own knowledge.

Example: What is cloud computing? Explain in simple words.

2. Reframing (Transformation) Prompt

Used to modify existing content by changing style, tone, perspective, or format.

Example: Rewrite this paragraph in a professional tone.

3. Few-Shot Prompt

Multiple examples are provided to teach the model the expected pattern or structure.

Example:

Example 1: Input: $2+3 \rightarrow 5$

Example 2: Input: $4+6 \rightarrow 10$

Task: Input: $7+8 \rightarrow ?$

4. Role-Based Prompt

The model is assigned a specific role to respond from that perspective.

Example: You are a cybersecurity expert. Explain how to prevent phishing attacks.

5. Instruction Prompt

Direct commands are given to the model specifying exactly what to do.

Example: List five advantages of artificial intelligence in bullet points.

6. Constraint-Based Prompt

The prompt includes rules such as word limit, tone, or format.

Example: Write a 100-word summary in formal language using bullet points.

7. Chain-of-Thought Prompt

Encourages the model to show step-by-step reasoning to reach the answer.

Example: Solve the problem step by step: If a train travels 60 km/h for 3 hours, how far does it go?

2.2 Generative AI

It is a type of artificial intelligence that can create new content—such as text, images, music, code, or videos—based on the data it has learned from. Instead of just analyzing or classifying information, generative AI *produces* completely new outputs that resemble human-created content.

Examples:

- ChatGPT generating human-like conversations
- DALL·E creating images from text prompts
- AI tools composing music or writing code

2.2.1 Types of modals

1. Language Models (LLMs)

- Generate human-like text.
- Used in chatbots, content generation, translation.
- Examples: GPT (ChatGPT), Google Gemini, LLaMA.

2. Image Generation Models

- Create images from text or enhance existing images.
- Examples: DALL·E, Midjourney, Stable Diffusion.

3. Audio & Music Generation Models

- Produce speech, sound effects, or music.
- Examples: Jukebox (OpenAI), Google AudioLM.

4. Video Generation Models

- Generate videos based on text or image inputs.
- Examples: Sora (by OpenAI), RunwayML.

5. Code Generation Models

- Write or debug computer code automatically.
- Examples: GitHub Copilot, Codex.

6. Multimodal Models

- Understand and generate multiple types of data (text, image, audio at the same time).
- Examples: GPT-4 with vision, Google Gemini Pro.

2.3 Large Language Model(LLM)

LLM (Large Language Model) is an advanced artificial intelligence model that is trained on massive amounts of text data to understand, generate, and interact using human-like language.

Examples of LLMs:

GPT-5 (what you are interacting with now), Google Gemini, Meta LLaMA, Claude

2.3.1 Key terms related to LLM are:

1. Token

A token is the smallest unit of text the model reads.

It can be a word, part of a word, or even a single character.

Example:

The sentence "*Artificial Intelligence*" may be split into tokens like:

["Artificial", "Intelligence"] or into smaller parts like ["Art", "ificial", "Intelli", "gence"]

2. Parameter

Parameters are the internal values or weights learned by the model during training.

They help the model understand language patterns.

More parameters = more intelligence and accuracy.

3. Prompt

The input or instruction given to the model to generate a response.

Example:

“Explain machine learning in simple language.”

4. Inference

The process of generating a response after the model has been trained.

Simply: Using the model to get an answer.

5. Fine-Tuning

Training an already trained model on specific data to make it better for a particular task or domain.

6. Temperature (in AI models)

A setting that controls creativity of the AI response.

- Low temperature (e.g., 0.2): More accurate, predictable answers
- High temperature (e.g., 0.9): More creative and varied answers

2.3.2 Applications of LLMs

1. Chatbots and Virtual Assistants

Used in customer service, healthcare support, and personal assistants like ChatGPT, Siri, and Alexa to answer queries and provide guidance.

2. Content Generation

LLMs can generate articles, blogs, emails, reports, social media posts, and creative writing automatically.

3. Code Generation and Debugging

Models like GPT can write code, fix errors, and explain programming concepts to developers.

4. Language Translation

Used in tools like Google Translate to convert text from one language to another with high accuracy.

5. Text Summarization

LLMs can summarize long articles, research papers, and legal documents into short and understandable content.

2.3.3 How To talk to LLM

To talk to an LLM effectively, you need to give clear, structured, and meaningful prompts. This process is called Prompt Engineering. The better you frame your prompt, the better the AI responds.

Example of a Good Prompt:

"Act as a career counsellor. Suggest three IT careers based on future demand and explain each in 50 words."

2.3.4 Limitations of LLMs

1. Lack of True Understanding

LLMs generate responses based on patterns in data, not real human understanding or consciousness.

2. Hallucination (False Information)

They may produce confident answers that are factually incorrect or completely made up.

3. No Real-Time Knowledge

LLMs cannot access the latest live information unless connected to updated databases.

4. Bias in Responses

Since LLMs are trained on internet data, they can reflect biases, stereotypes, or harmful content present in that data.

2.4 Playground

Playground is an interactive platform where users can experiment with AI models by typing prompts and observing how the model responds in real time.

It allows you to:

- Test different prompts
- Adjust parameters like temperature, token limit, etc.
- Understand how an AI model behaves
- Build and refine prompt engineering skills

2.4.1 Types of prompts in playground

System Prompt

This is given to the AI at the beginning to define its role, behaviour, tone, and rules. It guides how the AI should respond throughout the session.

Example: You are an expert doctor. Provide accurate answers in a formal tone.

User Prompt

This is the actual input or question given by the user to the AI for a response.

Example: What are the symptoms of diabetes?

2.4.2 Open Ai and Google Ai Studio Playground

OpenAI Playground

Developed by OpenAI. Primarily focused on text-based models like GPT. Offers advanced control

over parameters like temperature, frequency penalty, and stop sequences. Mainly supports text and code generation.

Google AI Studio (Gemini Playground)

Developed by Google. Supports both text and multimodal input (text, image, audio depending on the model). Offers structured prompts and role-based options. More focused on experimentation with Gemini models and real-world integrations

2.5 Prompt Evaluation Techniques

1. Output Accuracy Check

Evaluate whether the AI's response is factually correct and relevant to the prompt.

2. Consistency Testing

Give the same prompt multiple times or with slight variations to check if the AI gives consistent responses.

3. Coherence and Clarity Review

Examine if the response is logically structured, easy to understand, and follows a clear flow.

4. Relevance and Focus Analysis

Check whether the response stays on topic and directly addresses the user's intent rather than adding unnecessary information.

5. Compliance with Constraints

Evaluate if the AI follows specific instructions such as word limit, tone, format, or role as defined in the prompt.

2.6 LangChain

developers build LangChain is an open-source framework designed to help powerful applications using Large Language Models (LLMs). Instead of just generating text from prompts, LangChain allows LLMs to connect with external tools, databases, APIs, memory, and user data.

It helps in creating AI applications like chatbots, assistants, data analyzers, and automation systems with more control and customization.

Key Features:

- Supports prompt templates and chaining of multiple prompts
- Integrates memory to remember past user interactions
- Connects with tools like search engines, documents, or APIs
- Useful for building production-level AI applications

Example Use Cases:

Customer support bots, document analyzers, code assistants, and enterprise AI automation.

2.7 RAG (Retrieval-Augmented Generation)

RAG is a technique that improves the accuracy of language models by combining retrieval of real-world information with text generation.

Instead of relying only on the model's internal knowledge (which may be outdated or limited), RAG fetches relevant information from external sources (like databases or documents) during the prompt and uses it to generate accurate and updated output.

How it Works:

1. Retrieve: The system searches relevant content from a knowledge base.

2. Augment: This retrieved information is passed to the LLM.
3. Generate: The final answer is created using both the model's knowledge and the retrieved content.

Benefits:

- Provides factual and updated responses
- Reduces hallucination
- Ideal for enterprise, academic, or legal use cases

2.8 Notebook LM (Google AI Tool)

Notebook LM is an AI-powered research and learning assistant developed by Google. It is designed to help users interact with their own documents, notes, and study material.

You can upload files such as PDFs, Google Docs, or notes, and the AI will help you:

- Summarize the content
- Ask and answer questions based only on your documents
- Create study guides, insights, and reports

Purpose:

It works like a personalized AI tutor or analyst trained specifically on your own data.

2.9 Langflow

Langflow is a visual tool for building applications using Large Language Models (LLMs) without needing to write complex code. It is based on the LangChain framework and provides a drag-and-drop interface to create AI workflows.

What Langflow Does

It helps users build AI applications by visually connecting blocks such as:

- Prompt Templates
- LLM Models
- Memory
- Retrievers
- Agents
- Tools / API calls

Each block represents a component of an AI system, and you can simply connect them in a flowchart-style interface to design how the AI should work.

Key Features

- No-code / Low-code interface
- Drag-and-drop workflow
- Supports LangChain components
- Connect to OpenAI, Gemini, or local LLMs

2.10 Methodology

- Followed a step-by-step **learn–practice–implement** methodology during the training
- Practiced different **prompt engineering techniques** such as zero-shot, few-shot, role-based, and constraint-based prompts
- Understood how prompt structure and wording impact AI responses and output quality
- Gained hands-on experience with **LangChain and LangFlow** to design AI workflows
- Implemented components like prompts, models, memory, and retrievers in practical

CHAPTER 3

RESULTS AND DISCUSSION

3.1 Project Introduction

An AI-based web application titled “**URL Summarizer with Interactive AI Chat**” was developed during the training. The system extracts article content from a given URL, generates a concise summary using a generative AI model, and allows users to interact with the summary through an AI chat interface. Additional features such as keyword extraction and summary download enhance usability. The project demonstrates the practical application of AI and web technologies to reduce information overload.

3.2 Objectives

1. To automatically summarize web articles using AI.
2. To allow users to interact with AI based on the summary.
3. To reduce reading time and improve content understanding.

3.3 Summary Generation Results

The system successfully extracts textual data from web articles using the requests and BeautifulSoup libraries. It processes long articles and generates concise, meaningful summaries using Gemini AI.

Result: Summaries were found to reduce reading time by up to 80% on average.

Observation: The quality of the summary preserved the main context, important keywords, and conclusions of the articles

.

Example Outcome:

- Original text word count: 2500 words (Estimated reading time: 12 minutes)
- Generated summary: 300 words (Estimated time: 1.5 minutes)
- Reading time saved: approximately 10.5 minutes

3.4 AI Chat Interaction

A major feature of this project is the AI Chat module, which allows users to interact with the summarized content.

Result: Users could ask follow-up questions like:

- “What is the main issue discussed?”
- “What are the key benefits mentioned?”
- “Explain this article in simple words.”

Observation: This feature transforms the summarizer into a *conversational knowledge assistant*, enhancing understanding and personalization.

Impact: The AI chat makes the tool suitable for students, researchers, and professionals who want deeper insights without reading the full article.

3.5 PDF Generation Feature – Code Explanation

The project includes a feature that allows users to **download the AI-generated summary as a PDF report**. This report includes essential information such as the article title, URL, date, summary content, and estimated reading time saved. The PDF is generated using the reportlab library.

Code

```
import streamlit as st
import requests
from bs4 import BeautifulSoup
import google.generativeai as genai
from urllib.parse import urlparse

# =====
# 🗝️ Gemini Configuration
# =====
GEMINI_API_KEY = "AIzaSyBmtxsjCgRp9QK7U0nwvt7B84z_SNJ-YNM"
genai.configure(api_key=GEMINI_API_KEY)
MODEL = genai.GenerativeModel("gemini-2.5-flash")

# =====
# 🌐 Streamlit Setup
# =====
st.set_page_config(page_title="AI Summarizer + Chat", layout="centered")
st.title("🌐 URL Summarizer + AI Chat with Summary Context")
st.write("Paste an article URL to get a summary and then chat with AI based on that summary.")

# =====
# 🧠 Session State
# =====
for key in ["summary", "keywords", "title", "source", "chat_history"]:
    if key not in st.session_state:
        st.session_state[key] = None if key != "chat_history" else []

@st.cache_data(show_spinner=False)
def extract_article_data(url):
    headers = {"User-Agent": "Mozilla/5.0"}
    r = requests.get(url, headers=headers, timeout=10)
    r.raise_for_status()
    soup = BeautifulSoup(r.text, "html.parser")

    title = soup.title.string.strip() if soup.title else "Title not found"
    text = " ".join(p.get_text(strip=True) for p in soup.find_all("p"))
```

```

def generate_summary_and_keywords(text, length):
    text = text[:8000]
    style = {
        "Short": "4-5 bullet points",
        "Medium": "concise summary",
        "Detailed": "detailed explanation"
    }[length]

    prompt = f"""
Create a {style} summary and extract top 8 keywords.

Format exactly:
Summary:
- point

Keywords:
- keyword

Text:
{text}
"""

    return MODEL.generate_content(prompt).text

# =====
# 🗨 Chat
# =====
def chat_with_summary(question, summary):
    prompt = f"""
Answer strictly based on the summary below.

Summary:
{summary}

Question:
{question}

```

```

# =====
# 📄 UI Inputs
# =====
url = st.text_input("🔗 Enter article URL")
summary_length = st.selectbox("📄 Summary Length", ["Short", "Medium", "Detailed"])

if st.button("📄 Generate Summary"):
    if not url.startswith("http"):
        st.warning("⚠ Enter a valid URL")
    else:
        with st.spinner("🔄 Fetching article..."):
            title, source, article_text = extract_article_data(url)

            with st.spinner("✨ Generating summary..."):
                result = generate_summary_and_keywords(article_text, summary_length)

            summary, keywords = result.split("Keywords:")
            st.session_state.summary = summary.replace("Summary:", "").strip()
            st.session_state.keywords = keywords.strip()
            st.session_state.title = title
            st.session_state.source = source
            st.session_state.chat_history.clear()

# =====
# 📄 ALWAYS SHOW SUMMARY
# =====
if st.session_state.summary:
    st.markdown("---")
    st.subheader("📄 Article Details")
    st.write(f"***Title:** {st.session_state.title}")
    st.write(f"***Source:** {st.session_state.source}")

    st.subheader("📄 Summary")
    st.write(st.session_state.summary)

    st.download_button(
        "↓ Download Summary",

```

OUTPUT:

 **URL Summarizer + AI Chat with Summary Context**

Paste an article URL to get a summary and then chat with AI based on that summary.

 Enter article URL

 Summary Length

Medium

▼

 Generate Summary

 Generate Summary

 **Article Details**

Title: Python Tutorial | Learn Python Programming Language - Tpoint Tech

Source: www.tpointtech.com

 **Summary**

- Python is a popular, versatile programming language known for its simplicity, extensive libraries, and beginner-friendly design, supporting multiple programming paradigms.
- It features automatic memory management and is widely used across Data Science, Artificial Intelligence, Machine Learning, and Web Development.
- The text describes a comprehensive Python tutorial covering basics, advanced concepts like OOP, exception handling, and database connectivity.

 Download Summary

21

Chat with AI

Ask about the article

Ask



tell about python



Python is a popular, versatile programming language known for its simplicity, extensive libraries, and beginner-friendly design. It supports multiple programming paradigms and features automatic memory management. Python is widely used across Data Science, Artificial Intelligence, Machine Learning, and Web Development. There is a comprehensive Python tutorial available that covers basics, advanced concepts like Object-Oriented Programming (OOP), exception handling, and database connectivity.

3.6 Discussion & Insights

The project demonstrates the power of integrating **AI summarization, conversational intelligence, and documentation tools** into a single platform.

Efficiency: Reduced information overload and saved time.

User Engagement: The AI chat feature improved interactivity and increase user

Scalability: The system can be expanded to support file uploads, multilingual summaries, or domain-specific knowledge (health, finance, etc.).

CHAPTER 4

CONCLUSION AND FUTURE SCOPE

4.1 Future Scope

- Prompt engineering will become an essential skill as AI integrates into industries like healthcare, finance, education, cybersecurity, and automation.
- Advanced techniques like RAG, LangChain, and multimodal prompting will enable AI systems to work with real-time data and perform complex reasoning tasks.
- The rise of personalized AI agents will increase the demand for customized prompts that align with user-specific goals and business requirements.
- Future AI models will rely on prompt engineering to reduce hallucinations and improve factual accuracy and reliability.
- With the expansion of AI ethics and governance, prompt engineering will play a role in building responsible and bias-free AI systems.

4.2 Conclusion:

Prompt engineering has emerged as a transformative approach in the field of artificial intelligence, enabling more effective communication between humans and large language models. Rather than relying solely on traditional programming methods, prompt engineering uses natural language to guide AI systems and control their behavior, making AI more accessible and powerful. Through various prompt types and advanced frameworks like RAG, LangChain, and LangFlow, it enables the development of intelligent, context-aware, and personalized AI applications. Although challenges such as hallucinations and dependency on training data exist, continuous advancements in AI technology are steadily overcoming these limitations. In conclusion, prompt engineering is not just a technical skill but a strategic

References (IEEE Format)

- [1] T. Brown *et al.*, “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [2] OpenAI, “GPT-4 Technical Report,” OpenAI, 2023. [Online]. Available: <https://openai.com/research>
- [3] A. Vaswani *et al.*, “Attention Is All You Need,” in *Proc. 31st Conf. Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, USA, 2017.
- [4] OpenAI, “Introduction to Prompt Engineering,” OpenAI Documentation, 2024. [Online]. Available: <https://platform.openai.com/docs>